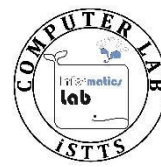




Institut Sains dan Teknologi Terpadu Surabaya

Jl. Ngagel Jaya Tengah 73 - 77, Surabaya 60284

Telp. (031) 5027920 Fax. (031) 5041509



Laboratorium	: L-204	Praktikum	: Web Services
Waktu	: Kamis, 15.45	Jurusan	: S1 – INF
Minggu Ke	: 3	Tanggal	: 18 Maret 2026
Materi	: Object Relational Mapping	Jenis Soal	: Materi & Tugas

MATERI (TOTAL: 35)

Anda dipromosikan menjadi *Lead Backend Engineer* di Gudang Serbaguna. Perusahaan baru saja menerapkan kebijakan *Flash Sale* otomatis dan standar penyimpanan gudang yang sangat ketat. Banyak keluhan dari manajer operasional karena data produk sering tidak konsisten: ada barang makanan yang kedaluwarsanya sudah lewat tapi masuk sistem, diskon *flash sale* yang tidak masuk akal, dan format SKU yang berantakan.

Tugas Anda adalah membuat satu *endpoint* API (tanpa relasi database) menggunakan **Sequelize ORM**. Validasi tidak hanya mengecek tipe data, tetapi juga mengecek **ketergantungan antar kolom** dan melakukan **kalkulasi otomatis di sisi server**.

Buatlah model Sequelize dengan nama Product dengan spesifikasi kolom berikut:

1. id : Integer, Primary Key, Auto Increment.
2. sku_code : String, Wajib diisi (Required), Unique.
3. name : String, Wajib diisi.
4. category : String, Wajib diisi.
5. base_price : Integer, Wajib diisi.
6. discount_percentage : Integer, Wajib diisi (Default 0).
7. final_price : Integer, Wajib diisi.
8. stock : Integer, Wajib diisi.
9. is_flash_sale : Boolean, Wajib diisi (Default false).

1. POST /api/products (Create Product)

Aturan Validasi & Business Logic:

- Nilai final_price wajib dihitung oleh sistem Backend dengan rumus: $\text{base_price} - (\text{base_price} * \text{discount_percentage} / 100)$.
- category: Hanya menerima "elektronik", "fashion", atau "makanan".
- sku_code: Formatnya wajib [KODE_KATEGORI]-[TAHUN]-[5_DIGIT_ANGKA].
 - Kategori "elektronik" wajib pakai kode ELK. (Contoh: ELK-2025-12345)
 - Kategori "fashion" wajib pakai kode FAS.
 - Kategori "makanan" wajib pakai kode MKN.
 - Tahun wajib menggunakan tahun saat ini (misal: 2025 atau 2026).
- base_price: Minimal Rp 10.000 dan **wajib kelipatan 500** (Contoh: 10500 valid, 10200 error).

- `is_flash_sale` & `discount_percentage`: Jika `is_flash_sale` bernilai `true`, maka `discount_percentage` **wajib** minimal 30 (`diskon >= 30%`).

Test Case	Request Body	Status Code	Expected Response (JSON)
Sukses: Normal Item	{ "sku_code": "FAS-2024-00001", "name": "Jaket Denim", "category": "fashion", "base_price": 150000, "discount_percentage": 10, "stock": 50, "is_flash_sale": false }	201	{ "status": "success", "data": { "final_price": 135000, "expiry_date": null } }
Sukses: Flash Sale & Food	{ "sku_code": "MKN-2024-99999", "name": "Keju Mozzarella", "category": "makanan", "base_price": 50000, "discount_percentage": 50, "stock": 100, "is_flash_sale": true, "expiry_date": "2026-12-31" }	201	{ "status": "success", "data": { "final_price": 25000, "expiry_date": "2026-12-31" } }
Error: SKU Mismatch	{ "sku_code": "FAS-2024-22222", "name": "Kabel Data", "category": "elektronik", "base_price": 25000, "discount_percentage": 0, "stock": 20, "is_flash_sale": false }	400	{ "status": "error", "message": "SKU code prefix does not match the category" }
Error: Harga Bukan Kelipatan	{ "sku_code": "FAS-2024-33333", "name": "Topi", "category": "fashion", "base_price": 12300, "discount_percentage": 0, "stock": 15, "is_flash_sale": false }	400	{ "status": "error", "message": "Base price must be a multiple of 500" }
Error: Flash Sale Diskon < 30%	{ "sku_code": "ELK-2024-44444", "name": "Keyboard", "category": "elektronik", "base_price": 100000, "discount_percentage": 10, "stock": 30, "is_flash_sale": true }	400	{ "status": "error", "message": "Flash sale items must have at least 30% discount" }

2. GET /api/products

Aturan Validasi & Business Logic:

- Wajib menerapkan Paginasi. Menerima Query Params: page (default 1) dan limit (default 10).
- Menerima filter opsional: ?category=... dan ?is_flash_sale=true/false
- Menerima sorting opsional: ?sort=cheapest (mengurutkan berdasarkan final_price termurah) atau ?sort=expensive.

Test Case	Request Query Params	Status Code	Expected Response (JSON)
Sukses: Get All Data	(Tidak ada)	200	{ "status": "success", "data": [{ "id": 1, "sku_code": "ELK-2026-12345", "name": "Keyboard Mechanical", "base_price": 500000, "...": "..." }] }
Sukses: Filter Multi-Kondisi	?category=fashion&is_flash_sale=true	200	{ "status": "success", "data": [{ "...": "hanya produk fashion yg sedang flash sale" }] }
Sukses: Filter & Sort	?category=elektronik&sort=cheapest	200	{ "status": "success", "data": [{ "...": "produk elektronik diurutkan dari base_price termurah" }] }
Sukses: Data Tidak Ditemukan	?category=makanan&is_flash_sale=true (Asumsi tidak ada data yg cocok)	200	{ "status": "success", "message": "No products found", "data": [] }

3. PUT /api/products/:id (Update Product)

Aturan Validasi & Business Logic:

- Mengecek ketersediaan ID.
- User TIDAK BOLEH mengubah sku_code, category, dan name. Jika dikirim di body, abaikan saja tapi jangan di-update.
- User hanya boleh mengubah base_price, discount_percentage, stock, dan is_flash_sale.
- Kalkulasi Ulang: Jika base_price atau discount_percentage diubah, sistem wajib menghitung ulang dan menyimpan final_price yang baru.
- Aturan Khusus Flash Sale: Jika barang sedang flash sale (is_flash_sale: true di database), dan user ingin mematikannya menjadi is_flash_sale: false, user TIDAK BISA melakukannya jika stock < 50.

Test Case	Request Body	Status Code	Expected Response (JSON)
Sukses: Update Harga	{ "base_price": 200000, "discount_percentage": 50 }	200	{ "status": "success", "message": "Product updated", "data": { "final_price": 100000 } }
Error: Data Tidak Ditemukan	ID = 999, { "stock": 10 }	404	{ "status": "error", "message": "Product not found" }
Error: Diskon Flash Sale	{ "discount_percentage": 10 } (Kondisi: barang saat ini is_flash_sale = true)	400	{ "status": "error", "message": "Flash sale items must have at least 30% discount" }
Error: Matikan Flash Sale (Stok Rendah)	{ "is_flash_sale": false } (Kondisi: is_flash_sale = true, stok saat ini = 10)	400	{ "status": "error", "message": "Cannot end flash sale if stock is less than 50" }

4. DELETE /api/products/:id (Delete Product)

Aturan Validasi & Business Logic:

- Mengecek ketersediaan ID.
- Barang HANYA BISA DIHAPUS jika memenuhi 2 syarat berikut secara bersamaan:
- stock barang harus 0.
- Barang sedang tidak dalam masa flash sale (is_flash_sale == false).
- Jika salah satu syarat tidak terpenuhi, tolak penghapusan.

Test Case	Kondisi Data Saat Ini (Di DB)	Status Code	Expected Response (JSON)
Sukses Delete	stock: 0 is_flash_sale: false	200	{ "status": "success", "message": "Product permanently deleted" }
Error: Data Tidak Ada	ID = 9999 tidak ada di database	404	{ "status": "error", "message": "Product not found" }
Error: Stok Masih Ada	stock: 5 is_flash_sale: false	400	{ "status": "error", "message": "Cannot delete product. Stock must be 0." }
Error: Flash Sale Aktif	stock: 0 is_flash_sale: true	400	{ "status": "error", "message": "Cannot delete product. Flash sale is currently active." }

**DILARANG MENGGUNAKAN MATERI YANG BELUM DIAJARKAN PADA MINGGU INI
JIKA MELANGGAR MAKA NILAI MATERI : 0**

PERHATIKAN KETENTUAN DIBAWAH :

- Highlight kriteria yang dikerjakan dengan warna kuning dan kumpulkan word beserta dengan file materi, apabila tidak dikumpulkan maka materi tidak akan diperiksa.
- Akan ada pengurangan nilai sebesar -5 untuk setiap kriteria yang dihighlight namun tidak dikerjakan.
- **MENCONTEK = Nilai MOD 2**

MATERI : 35

SCORE	KRITERIA
0/10	Endpoint POST berjalan dengan baik beserta validasi (@2 per test case)
0/8	Endpoint GET berjalan dengan baik beserta validasi (@2 per test case)
0/8	Endpoint PUT berjalan dengan baik beserta validasi (@2 per test case)
0/8	Endpoint DELETE berjalan dengan baik beserta validasi (@2 per test case)
-35/1	Menggunakan sequelize model
Total : 35	

TUGAS (TOTAL = 25)

Lanjutkan materi.

Pada sesi Materi, Anda telah berhasil membangun API Katalog Produk dengan validasi harga dan kategori. Sekarang, CTO GudangZ memberikan tantangan baru. Gudang fisik perusahaan kini dibagi menjadi beberapa **Zona (Zone/Rak Gudang)**. Setiap zona memiliki **batas maksimal kapasitas stok barang (max_capacity)**.

Tugas Anda adalah memodifikasi sistem Produk yang sudah ada agar terintegrasi dengan sistem Zona. Karena *Database Architecture* saat ini masih menggunakan konsep terpisah, **Anda dilarang keras menggunakan fitur Relasi/Asosiasi bawaan Sequelize (hasMany, belongsTo, dll)**. Semua pengecekan apakah Zona itu ada, serta perhitungan apakah kapasitas Zona masih muat menampung stok barang baru, **wajib dilakukan secara manual (melalui logika koding Controller)** di *endpoint* Anda.

Selain itu, karena data inventaris sangat rahasia, data produk kini **tidak boleh dihapus permanen (wajib Soft Delete)** dan data yang tampil wajib menggunakan **Paginasi**.

1. Spesifikasi Perubahan Model Database (Tanpa Asosiasi/Relasi)

A. Buat Model Baru: Zone (Zona Gudang)

Buat tabel baru dengan kolom berikut:

1. id : Integer, Primary Key, Auto Increment.
2. zone_code : String, Wajib diisi (Required), Unique (Contoh: "ZON-A").
3. max_capacity : Integer, Wajib diisi (Kapasitas maksimal kuantitas barang di zona ini).
4. current_load : Integer, Wajib diisi, Default 0 (Kapasitas terpakai saat ini).

B. Modifikasi Model Lama: Product

1. Tambahkan kolom baru: zone_code (String, Wajib diisi). Ini berfungsi sebagai referensi manual ke tabel Zone.
2. **Wajib mengaktifkan opsi paranoid: true** pada konfigurasi model Sequelize Product agar fitur *Soft Delete* (kolom deletedAt) aktif secara otomatis.

(Catatan: Semua kolom lama seperti sku_code, name, category, base_price, stock tetap ada dan validasi dari sesi materi tetap berlaku).

1. POST /api/zones (Create Zone)

- Validasi: zone_code wajib diawali teks "ZON-" dan diikuti persis 1 huruf kapital (Contoh Valid: "ZON-A", "ZON-B". Invalid: "ZONE-1", "ZON-AA").
- Validasi: max_capacity wajib angka minimal 50. Kolom current_load tidak boleh dikirim oleh user (otomatis 0).

Test Case	Request Body	Status Code	Expected Response (JSON)
Sukses Create	{ "zone_code": "ZON-A", "max_capacity": 500 }	201	{ "status": "success", "data": { "zone_code": "ZON-A", "current_load": 0 } }

			}
Error: Format Invalid	{ "zone_code": "ZONE-X", "max_capacity": 100 }	400	{ "status": "error", "message": "Invalid zone_code format" }

2. POST /api/products (Create Product & Manual Capacity Check)

- User kini wajib mengirimkan zone_code di dalam Request Body.
- Validasi Relasi Lintas Tabel (Manual):
 - Ambil nilai zone_code dari body, cari di tabel Zone. Jika tidak ada, kembalikan error 404 (Zone not found).
 - Jika Zona ditemukan, cek kapasitasnya: Apakah $\text{Zone.current_load} + \text{Product.stock}$ melebihi Zone.max_capacity ? Jika melebihi, Tolak / Error.
 - Jika kapasitas aman: Simpan data Product ke database, lalu WAJIB update nilai current_load di tabel Zone (+ ditambahkan dengan stok barang baru).

Test Case	Request Body (Contoh)	Status Code	Expected Response (JSON)
Sukses Create	{ "sku_code": "ELK-2026-00001", "name": "Laptop", "category": "elektronik", "base_price": 5000000, "stock": 100, "zone_code": "ZON-A" }	201	{ "status": "success", "message": "Product saved and Zone capacity updated" }
Error: Zona Tidak Ada	{ "sku_code": "ELK-2026-00002", "stock": 50, "zone_code": "ZON-Z", "...": "..." }	404	{ "status": "error", "message": "Target zone not found" }
Error: Kapasitas Overload	{ "sku_code": "ELK-2026-00003", "stock": 450, "zone_code": "ZON-A", "...": "..." } (Asumsi ZON-A sisa kapasitas hanya 400)	400	{ "status": "error", "message": "Zone capacity exceeded. Cannot store this much stock." }

3. GET /api/products (Read All items with Pagination)

- Wajib Paginasi: Menerima Query Params page (default 1) dan limit (default 5).
- Data JSON wajib dibungkus dengan metadata (total_data, total_pages, current_page) dan data.
- Aturan Soft Delete: Data produk yang sudah dihapus (soft-deleted) TIDAK BOLEH muncul di endpoint ini.

Test Case	Request Query Params	Status Code	Expected Response (JSON)
Sukses Paginasi Default	(Tidak ada)	200	{ "status": "success", "metadata": { "total_data": 12, } }

			<pre> "total_pages": 3, "current_page": 1 }, "data": [{ "...": "..." }] } </pre>
Sukses Custom Page	?page=2&limit=10	200	<pre> { "status": "success", "metadata": { "total_data": 12, "total_pages": 2, "current_page": 2 }, "data": [{ "...": "..." }] } </pre>

4. PUT /api/products/:id (Update Stock & Kalkulasi Selisih Zona)

- User diizinkan meng-update nilai stock barang. User dilarang meng-update zone_code.
- Logika Kalkulasi Selisih (Ngelotok Logika): Jika user mengubah stock, Anda harus menghitung selisihnya terhadap stok lama di database.
- Skenario A (Stok Nambah): Stok awal 100, di-update jadi 150. Berarti ada tambahan 50 ke Zona. Wajib cek apakah tambahan 50 ini bikin Zona overload! Jika ya, tolak. Jika aman, simpan produk dan tambah beban Zona sebesar 50.
- Skenario B (Stok Berkurang): Stok awal 100, di-update jadi 80. Berarti beban Zona berkurang 20. Simpan produk dan kurangi beban Zona sebesar 20.

Test Case	Request Body	Status Code	Expected Response (JSON)
Sukses Update (Stok Berkurang)	<pre> { "stock": 80 } </pre> (Awalnya 100. Beban zona akan -20)	200	<pre> { "status": "success", "message": "Stock updated and Zone load adjusted (-20)" } </pre>
Sukses Update (Stok Nambah & Aman)	<pre> { "stock": 150 } </pre> (Awalnya 100. Beban zona akan +50)	200	<pre> { "status": "success", "message": "Stock updated and Zone load adjusted (+50)" } </pre>
Error: Update Bikin Overload	<pre> { "stock": 900 } </pre> (Nambah 800 stok baru, kapasitas zona tidak muat)	400	<pre> { "status": "error", "message": "Updating stock causes zone capacity overload" } </pre>

5. DELETE /api/products/:id (Soft Delete & Restorasi Kapasitas)

- Wajib mengecek apakah data produk dengan ID tersebut ada dan belum terhapus.
- Wajib menggunakan Soft Delete (paranoid: true), dilarang menghapus permanen.

- Logika Restorasi Kapasitas: Jika produk dihapus (dikeluarkan dari gudang), maka ruang kosong pada Zona tersebut akan bertambah. Anda wajib mengurangi nilai `current_load` pada tabel `Zone` sesuai dengan jumlah stock barang yang dihapus tersebut.

Test Case	Request Method & URL	Status Code	Expected Response (JSON)
Sukses Soft Delete	DELETE /api/products/1 (Barang ID 1 punya stok 50)	200	{ "status": "success", "message": "Product soft deleted. Zone load decreased by 50." }
Error: Data Tidak Ada	DELETE /api/products/999	404	{ "status": "error", "message": "Product not found" }

**DILARANG MENGGUNAKAN KONSEP MAUPUN MATERI YANG BELUM DIAJARKAN
JIKA MELANGGAR MAKA NILAI TUGAS: 0
WAJIB HAPUS NODE MODULES ATAU NILAI TUGAS: 0**

PERHATIKAN KETENTUAN DIBAWAH :

- Highlight kriteria yang dikerjakan dengan warna kuning dan kumpulkan word beserta dengan file tugas, apabila tidak dikumpulkan maka tugas tidak akan diperiksa.
- Akan ada pengurangan nilai sebesar -5 untuk setiap kriteria yang dihighlight namun tidak dikerjakan.
- **MENCONTEK = Nilai MOD 2**

TUGAS : 25

SCORE	KRITERIA
0/4	Endpoint POST Zone berjalan dengan baik beserta validasi
0/4	Endpoint POST Product berjalan dengan baik beserta validasi
0/4	Pagination dan limit GET berjalan dengan baik
0/4	Endpoint PUT berjalan dengan baik beserta validasi
0/4	Endpoint DELETE berjalan dengan baik beserta validasi
0/4	Semua validasi dari materi berjalan dengan baik
-25/1	Menggunakan sequelize model
Total : 25	

Menyetujui

Mengetahui

Penyusun Soal

(Mikhael Setiawan, S.Kom.,
M.Kom.)
Koordinator Kuliah

(Grace Levina Dewi, M.Kom.)
Koordinator Laboratorium

(Vincentius Jason Tjendika)
Asisten